

Project Report on
VR Flight Simulator

Submitted in Partial fulfillment of requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

COMPUTER SCIENCE AND ENGINEERING (DATA SCIENCE)

By

B. SAKETH REDDY	20BD1A6707
B. ROHIT	20BD1A6706
GANUMALA LAVITRA LALIT	20BD1A6721
ATHARVA RAJE	20BD1A6704

Under the guidance of

Dr. K. VISHAL REDDY
Assistant Professor
Department of CSE (DATA SCIENCE)



DEPARTMENT OF CSE (DATA SCIENCE)

KESHAV MEMORIAL INSTITUTE OF TECHNOLOGY
(AN AUTONOMOUS INSTITUTION)

Accredited by NBA & NAAC, Approved by AICTE, Affiliated to JNTUH.
Narayanaguda, Hyderabad, Telangana-29
2023-24



**KESHAV MEMORIAL INSTITUTE OF
TECHNOLOGY**

(AN AUTONOMOUS INSTITUTION)

Accredited by NBA & NAAC, Approved by AICTE, Affiliated to JNTUH.
Narayanaguda, Hyderabad, Telangana-29



DEPARTMENT OF CSE (DATASCIENCE)

CERTIFICATE

This is to certify that this is a bonafide record of the project report titled **“VR
FLIGHT SIMULATOR”** which is being presented as the Project report by

B. SAKETH REDDY	20BD1A6707
B. ROHIT	20BD1A6706
GANUMALA LAVITRA LALIT	20BD1A6721
ATHARVA RAJE	20BD1A6704

In partial fulfillment for the award of the degree of Bachelor of Technology in
Computer Science and Engineering (Data Science) affiliated to the Jawaharlal Nehru
Technological University, Hyderabad.

Internal Guide
(Dr. K. Vishal Reddy)

Head of Department
(Mr. K. Anil Kumar)

Submitted for Viva Voce Examination held on _____

External Examiner

Vision of KMIT

- To be the fountainhead in producing highly skilled, globally competent engineers.
- Producing quality graduates trained in the latest software technologies and related tools and striving to make India a world leader in software products and services.

Mission of KMIT

- To provide a learning environment that inculcates problem solving skills, professional, ethical responsibilities, lifelong learning through multi modal platforms and prepares students to become successful professionals.
- To establish an industry institute Interaction to make students ready for the industry.
- To provide exposure to students on the latest hardware and software tools.
- To promote research-based projects/activities in the emerging areas of technology convergence.
- To encourage and enable students to not merely seek jobs from the industry but also to create new enterprises.
- To induce a spirit of nationalism which will enable the student to develop, understand India's challenges and to encourage them to develop effective solutions.
- To support the faculty to accelerate their learning curve to deliver excellent service to students.

Vision of CSE (Data Science) Department

The vision of the department is to impart quality education, inculcate professionalism and enhance the problem-solving skills of the students in the domain of DATA SCIENCE with a focus to make them industry ready, involve in possible areas of research, to pursue and have continual professional growth.

Mission of CSE (Data Science) Department

- To equip the students with strong fundamental concepts, analytical capability, programming, and problem-solving skills.
- To create an academic environment conducive for higher learning through faculty training, self-learning, sound academic practices and research endeavors.
- To provide opportunities to promote organizational and leadership skills in students through various co-curricular and extra – curricular activities.
- To make the students industry ready and to enhance their employability through training and internships.
- To improve department industry collaboration through interaction including participation in professional society activities, guest lecturers and industrial visit.

PROGRAM OUTCOMES (POs)

1. **Engineering Knowledge:** Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
2. **Problem Analysis:** Identify formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
3. **Design/Development of solutions:** Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
4. **Conduct Investigations of Complex problems:** Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
5. **Modern Tool Usage:** Create select, and, apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.
6. **The Engineer and Society:** Apply reasoning informed by contextual knowledge to societal, health, safety. Legal und cultural issues and the consequent responsibilities relevant to professional engineering practice.

7. **Environment and Sustainability:** Understand the impact of the professional engineering solutions in societal and environmental contexts and demonstrate the knowledge of, and need for sustainable development.
8. **Ethics:** Apply ethical principles and commit to professional ethics and responsibilities and norms of engineering practice.
9. **Individual and Team Work:** Function effectively as an individual, and as a member or leader in diverse teams and in multidisciplinary settings.
10. **Communication:** Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
11. **Project Management and Finance:** Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
12. **Life-Long Learning:** Recognize the need for and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

PROGRAM SPECIFIC OUTCOMES (PSOs)

PSO1: An ability to analyze the common business functions to design and develop appropriate Information Technology solutions for social upliftments.

PSO2: Shall have expertise on the evolving technologies like Python, Machine Learning, Deep learning, IOT, Data Science, Full stack development, Social Networks, Cyber Security, Mobile Apps, CRM, ERP, Big Data, etc.

PROGRAM EDUCATIONAL OBJECTIVES (PEOs)

PEO1: Graduates will have successful careers in computer related engineering fields or will be able to successfully pursue advanced higher education degrees.

PEO2: Graduates will try and provide solutions to challenging problems in their profession by applying computer engineering principles.

PEO3: Graduates will engage in life-long learning and professional development by rapidly adapting to the changing work environment.

PEO4: Graduates will communicate effectively, work collaboratively, and exhibit high levels of professionalism and ethical responsibility.

PROJECT OUTCOMES

P1: To create a simulation of a live flight.

P2: To create a market ready flight simulation model that can be used for training.

P3: Provide a live application for making the product available.

P4: Understanding and implementing advanced concepts of VR.

MAPPING PROJECT OUTCOMES WITH PROGRAM OUTCOMES

PO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12
P1	M	L	H	H	-	L	H	L	H	H	-	H
P2	H	H	M	H	L	-	M	-	H	H	H	H
P3	H	H	M	H	L	-	H	L	H	H	H	H
P4	H	H	M	H	L	L	H	-	H	H	H	H

L – LOW

M –MEDIUM

H– HIGH

**PROJECT OUTCOMES MAPPING PROGRAM
SPECIFIC OUTCOMES**

PSO	PSO1	PSO2
P1	L	M
P2	M	L
P3	H	M
P4	L	L

**PROJECT OUTCOMES MAPPING PROGRAM
EDUCATIONAL OBJECTIVES**

PEO	PEO1	PEO2	PEO3	PEO4
P1	M	M	L	M
P2	M	M	M	L
P3	M	H	M	L
P4	L	M	H	L

DECLARATION

We hereby declare that the results embodied in the dissertation entitled **“VR FLIGHT SIMULATOR”** has been carried out by us together during the academic year 2023-24 as a partial fulfillment of the award of the B.Tech degree in CSE (Data Science) from Keshav Memorial Institute of Technology, Autonomous Institute, affiliated to JNTUH, Accredited by NAAC and NBA, Hyderabad. We have not submitted this report to any other university or organization for the award of any other degree.

B. SAKETH REDDY (20BD1A6707)

B. ROHIT (20BD1A6706)

GANUMALA LAVITRA LALIT (20BD1A6721)

ATHARVA RAJE (20BD1A6704)

ACKNOWLEDGEMENT

We take this opportunity to thank all the people who have rendered their full support to our project work. We render our thanks to **Dr. B L Malleswari**, Principal who encouraged us to do the Project.

We are grateful to **Mr. Neil Gogte**, Founder & Director, **Mr. S. Nitin**, Director for facilitating all the amenities required for carrying out this project.

We express our sincere gratitude to **Ms. Deepa Ganu**, Director Academic for providing an excellent environment in the college.

We are also thankful to **Mr. K. Anil Kumar**, Head of the Department for providing us with time to make this project a success within the given schedule.

We are also thankful to our guide **Dr. K. Vishal Reddy**, for his valuable guidance and encouragement given to us throughout the project work.

We would like to thank the entire CSE (Data Science) Department faculty, who helped us directly and indirectly in the completion of the project.

We sincerely thank our friends and family for their constant motivation during the project work.

B. SAKETH REDDY (20BD1A6707)

B. ROHIT (20BD1A6706)

GANUMALA LAVITRA LALIT (20BD1A6721)

ATHARVA RAJE (20BD1A6704)

ABSTRACT

The Virtual Reality (VR) Flight Simulator project aims to deliver an immersive and realistic flying experience through the integration of cutting-edge VR technologies. Leveraging the Unity game development platform, this project focuses on creating a dynamic and engaging environment for pilot training, entertainment, and research purposes. The simulator incorporates realistic flight dynamics, 3D modeling of aircraft and landscapes, and a user-friendly interface within the virtual cockpit. Key features include interactive controls through motion controllers, a heads-up display providing essential flight information, and a variety of training scenarios to cater to different skill levels. The project emphasizes a user-centered design approach, optimizing the user experience to mitigate simulator sickness and enhance immersion. In addition to the core simulation elements, the VR Flight Simulator project explores multiplayer and collaborative features, enabling shared experiences within the virtual airspace. The development process includes rigorous testing, iteration based on user feedback, and documentation of design decisions and codebase. As VR technologies continue to evolve, this project contributes to the growing field of VR flight simulation, offering a platform for realistic training scenarios, skill development, and collaborative learning. The project's modular and extensible design allows for future updates, ensuring compatibility with emerging VR hardware and software advancements. Ultimately, the VR Flight Simulator project aspires to provide a versatile and enjoyable VR experience for aviation enthusiasts, aspiring pilots, and researchers alike.

The Flight Simulator project is aimed at developing a flight simulator in UNITY 3D. The project aims to provide a realistic flight simulation experience to the users, where they can experience flying different aircraft types in various environments.

LIST OF ABBREVIATIONS

VR	Virtual Reality
AR	Augmented Reality
HMD	Head Mounted Display
MR	Mixed Reality
XR	Extended Reality

LIST OF DIAGRAMS

S. No	Name of the Diagram	Page No.
1.	Use Case Diagram	16
2.	Class Diagram	17
3.	Terrain	20
4.	Simulation Clips	21
5.	Hangar	26
6.	Oculus UI	26
7.	Flight Dynamics	27
8.	Flight Runaway	28

CONTENTS

<u>DESCRIPTION</u>	<u>PAGE</u>
CHAPTER – 1	
1. INTRODUCTION	1-8
1.1 Purpose of the project	1
1.2 Scope of the Project	2
1.3 Virtual Reality	2
CHAPTER – 2	
2. LITERATURE SURVEY	9-11
CHAPTER - 3	
3. SOFTWARE REQUIREMENT SPECIFICATION	12-13
3.1 Functional Requirements	12
3.2 User Requirement Analysis	12
3.3 Input Design	13
3.4 Input Types	13
3.5 Output Design	13
CHAPTER – 4	
4.SYSTEM DESIGN	14-17
4.1 Introduction to UML	14
4.2 UML Design	16
4.2.1 Use case Diagram	16

4.2.2 Class Diagram	17
CHAPTER – 5	
5. METHODOLOGIES	18-21
5.1 Problems with Existing System	18
5.2 Proposed System	20
CHAPTER - 6	
6. IMPLEMENTATION	22-25
CHAPTER – 7	
7. SCREENSHOTS	26-28
CHAPTER - 8	
8. TESTING	29-34
8.1 Introduction to Testing	24
8.2 Software Testing Life Cycle	25
8.3 White Box Testing	31
8.4 What is Black Box Testing?	33
CHAPTER - 9	
9. FUTURE ENHANCEMENTS	35-36
CHAPTER - 10	
10. CONCLUSION	37- 38
CHAPTER - 11	
11. REFERENCES	39-40

CHAPTER - 1

1. INTRODUCTION

1.1 Purpose of the project

The development of flight simulators has always been driven by the pursuit of realism and the desire to provide both entertainment and educational value to users. These simulators have been pivotal in training pilots, enhancing aviation safety, and satisfying the curiosity of aviation enthusiasts.

The Flight Simulator project emerged from this rich tradition and is motivated by several key factors:

Technological Advancements: Over the years, the rapid progress in technology, particularly in the field of game development and computer graphics, has allowed for the creation of increasingly realistic and immersive simulation environments. These advancements have opened exciting possibilities for creating highly realistic flight simulations that can mirror the complexities of real-world aviation.

Realistic Experience: The emphasis on high-fidelity graphics, realistic physics, and intuitive controls within the project ensures that users can immerse themselves in an authentic flying experience. This level of realism not only caters to the gaming community but also attracts aviation enthusiasts who appreciate the intricate details of aircraft operation and aviation enthusiasts who seek an authentic flying experience.

Versatility: The project's focus on Unity 3D integration brings versatility to the simulator. Users can expect a wide range of aircraft and environments to explore, from casual flying in calm skies to navigating challenging weather conditions.

In summary, the Flight Simulator project is motivated by the convergence of technology, accessibility, education, realism, and versatility. By harnessing the power of modern game development tools and technologies, it aspires to offer a comprehensive and immersive flight simulation experience that caters to both aviation enthusiasts and those with a passion for gaming and aviation industries.

1.2 Scope of the Project:

The Flight Simulator project is aimed at developing a flight simulator in UNITY 3D that will be integrated with the MERN stack (MongoDB, Express, React, and Node.js). The project aims to provide a realistic flight simulation experience to the users, where they can experience flying different aircraft types in various environments.

Realistic Flight Simulation: The project's primary scope is to develop a highly realistic flight simulation using Unity 3D, with a focus on delivering an immersive experience that accurately mimics the complexities of real-world aviation.

Cross-Platform Accessibility: The flight simulator will be accessible across various devices and web browsers, allowing a wide audience to enjoy the experience without the need for specialized installations.

User Registration and Authentication: The scope includes implementing a secure user registration system, ensuring that only registered users can access and play the game, with user data stored in a MongoDB database.

Client-Server Communication: The project involves establishing an efficient and low-latency communication system between the Unity client and the MERN stack server, enabling smooth interactions and responsive gameplay.

1.2 VIRTUAL REALITY

Virtual Reality (VR) has a rich and varied history, marked by technological advancements, shifts in industry focus, and evolving consumer interest. From its conceptualization in the mid-20th century to its widespread adoption in various sectors today, VR has undergone a remarkable journey, transforming the way we interact with digital content and shaping the future of immersive experiences.

The concept of virtual reality can be traced back to the 1950s and 1960s when pioneers like Morton Heilig envisioned immersive experiences that could transport users to virtual worlds.

Heilig's invention, the Sensorama, was one of the earliest attempts at creating a multi-sensory VR experience, incorporating visuals, sound, vibration, and even smells.

In the 1960s and 1970s, researchers and engineers began experimenting with computer-generated graphics and interactive simulations, laying the groundwork for modern VR technology. Ivan Sutherland's "Sword of Damocles" system, developed in 1968, is often cited as one of the first true VR headsets, although its bulky size and limited capabilities made it impractical for widespread use.

The 1980s and 1990s saw the commercialization of VR technology, with companies like Atari, Nintendo, and Sega releasing VR-related products and arcade games. However, these early attempts at consumer VR were met with limited success due to technical limitations, high costs, and a lack of compelling content.

One notable exception was the Virtuality arcade system, introduced in the early 1990s, which offered immersive VR experiences through stereoscopic headsets and motion-tracking controllers. Despite its novelty, Virtuality failed to gain widespread traction, and interest in VR waned as the technology struggled to deliver on its promise of immersive experiences.

The early 2000s marked a period of resurgence for VR, driven by advancements in computing power, graphics technology, and motion tracking. In 2012, Oculus VR, founded by Palmer Luckey, launched a Kickstarter campaign for the Oculus Rift, a high-quality VR headset designed for gaming and entertainment. The campaign's success ignited renewed interest in VR among developers, investors, and consumers, sparking a wave of innovation and investment in the industry.

In 2016, consumer VR reached a major milestone with the commercial release of the Oculus Rift, HTC Vive, and PlayStation VR, bringing high-quality VR experiences to the mass market. These headsets offered immersive visuals, precise motion tracking, and intuitive controls, ushering in a new era of VR gaming, entertainment, and storytelling.

Since then, VR has expanded beyond gaming to encompass a wide range of applications across various industries, including:

1. Entertainment and Gaming: VR has revolutionized the gaming industry, offering players immersive experiences that transport them to fantastical worlds, realistic simulations, and interactive narratives. From action-packed shooters to serene exploration games, VR has redefined what's possible in gaming, providing unparalleled levels of immersion and interactivity.

2. Education and Training: VR has emerged as a powerful tool for education and training, enabling students to explore virtual environments, conduct experiments, and engage in hands-on learning experiences. In fields such as medicine, engineering, and aviation, VR simulations are used to train professionals, simulate complex scenarios, and improve learning outcomes.

3. Healthcare and Therapy: VR is increasingly being used in healthcare for therapeutic purposes, including pain management, rehabilitation, and mental health treatment. VR simulations can help patients alleviate pain, overcome phobias, and manage stress and anxiety by immersing them in calming environments or guiding them through exposure therapy exercises.

4. Architecture and Design: VR is transforming the way architects and designers visualize and present their projects, allowing them to create immersive 3D environments that clients can explore in real time. VR simulations enable stakeholders to experience architectural designs at scale, visualize spatial layouts, and provide feedback on design decisions, leading to more informed and collaborative design processes.

5. Military and Defense: VR is utilized by the military for training purposes, allowing soldiers to practice tactical maneuvers, weapon handling, and mission planning in realistic virtual environments. VR simulations can replicate combat scenarios, simulate battlefield conditions, and train troops for a variety of operational scenarios, helping to improve readiness and effectiveness on the battlefield.

Currently, standard virtual reality systems use either virtual reality headsets or multi-projected environments to generate some realistic images, sounds and other sensations that simulate a user's physical presence in a virtual environment.

The effect is commonly created by VR headsets consisting of a head-mounted display with a small screen in front of the eyes, but can also be created through specially designed rooms with multiple large screens. Virtual reality typically incorporates auditory and video feedback, but may also allow other types of sensory and force feedback through haptic technology.

One method by which virtual reality can be realized is simulation-based virtual reality. Driving simulators, for example, give the driver on board the impression of actually driving an actual vehicle by predicting vehicular motion caused by driver input and feeding back corresponding visual, motion and audio cues to the driver.

With avatar image-based virtual reality, people can join the virtual environment in the form of real video as well as an avatar. One can participate in the 3D distributed virtual environment as form of either a conventional avatar or a real video. Users can select their own type of participation based on the system capability.

In projector-based virtual reality, modeling of the real environment plays a vital role in various virtual reality applications, including robot navigation,

construction modeling, and airplane simulation. Image-based virtual reality systems have been gaining popularity in computer graphics and computer vision communities.

In generating realistic models, it is essential to accurately register acquired 3D data; usually, a camera is used for modeling small objects at a short distance. Desktop-based virtual reality involves displaying a 3D virtual world on a regular desktop display without use of any specialized VR positional tracking equipment. Many modern first-person video games can be used as an example, using various triggers, responsive characters, and other such interactive devices to make the user feel as though they are in a virtual world.

A common criticism of this form of immersion is that there is no sense of peripheral vision, limiting the user's ability to know what is happening around them.

A head-mounted display (HMD) more fully immerses the user in a virtual world. A virtual reality headset typically includes two small high-resolution OLED or LCD monitors which provide separate images for each eye for stereoscopic graphics rendering a 3D virtual world, a binaural audio system, positional and rotational real-time head tracking for six degrees of movement. Options include motion controls with haptic feedback for physically interacting within the virtual world in an intuitive way with little to no abstraction and an omnidirectional treadmill for more freedom of physical movement allowing the user to perform locomotive motion in any direction.

Augmented reality (AR) is a type of virtual reality technology that blends what the user sees in their real surroundings with digital content generated by computer

software. The additional software-generated images with the virtual scene typically enhance how the real surroundings look in some way. AR systems layer virtual information over a camera live feed into a headset or smart glasses or through a mobile device giving the user the ability to view three-dimensional images.

The Virtual Reality Modelling Language (VRML), first introduced in 1994, was intended for the development of "virtual worlds" without dependency on headsets. The Web3D consortium was subsequently founded in 1997 for the development of industry standards for web-based 3D graphics.

The consortium subsequently developed X3D from the VRML framework as an archival, open-source standard for web-based distribution of VR content. WebVR is an experimental JavaScript application programming interface (API) that provides support for various virtual reality devices, such as the HTC Vive, Oculus Rift, Google Cardboard or OSVR, in a web browser.

Virtual reality is most used in entertainment applications such as video games, 3D cinema, amusement park rides including dark rides and social virtual worlds. Consumer virtual reality headsets were first released by video game companies in the early-mid 1990s. Beginning in the 2010s, next-generation commercial tethered headsets were released by Oculus (Rift), HTC (Vive) and Sony (PlayStation VR), setting off a new wave of application development. 3D cinema has been used for sporting events, fine art, music videos and short films.

Since 2015, roller coasters and theme parks have incorporated virtual reality to match visual effects with haptic feedback. VR not only fits the trend of the digital industry but also enhances the film's visual effect. The film gives the audience more ways to interact through VR technology.

The Virtual Reality Modelling Language (VRML), first introduced in 1994, was intended for the development of "virtual worlds" without dependency on headsets.[64] The Web3D consortium was subsequently founded in 1997 for the development of industry standards for web-based 3D graphics.

The consortium subsequently developed X3D from the VRML framework as an archival, open-source standard for web-based distribution of VR content.[65] WebVR is an experimental JavaScript application programming interface (API) that provides support for various virtual reality devices, such as the HTC Vive, Oculus Rift, Google Cardboard or OSVR, in a web browser.

Virtual reality is most used in entertainment applications such as video games, 3D cinema, amusement park rides including dark rides and social virtual worlds.

Beginning in the 2010s, next-generation commercial tethered headsets were released by Oculus (Rift), HTC (Vive) and Sony (PlayStation VR), setting off a new wave of application development. 3D cinema has been used for sporting events, fine art, music videos and short films. Since 2015, roller coasters and theme parks have incorporated virtual reality to match visual effects with haptic feedback. VR not only fits the trend of the digital industry but also enhances the film's visual effect.

CHAPTER - 2

2. LITERATURE SURVEY

Researchers have explored various development platforms and tools for creating VR flight simulator projects. Unity and Unreal Engine are commonly mentioned, and discussions often revolve around the advantages and limitations of each.

Literature discusses the integration of VR hardware components, such as head-mounted displays (HMDs), motion controllers, and haptic feedback devices, into flight simulator projects. Evaluations focus on the impact of these technologies on user immersion and training effectiveness. The realism of flight dynamics in VR flight simulators is a recurrent theme.

Research often addresses the modeling of aerodynamics, weather conditions, and other environmental factors to enhance the fidelity of the simulator. A significant portion of the literature examines the use of VR flight simulators for training purposes. This includes pilot training, skill development, and proficiency testing. Researchers explore the effectiveness of VR simulations compared to traditional training methods.

Human factors play a crucial role in the design of VR flight simulator projects. Literature often covers aspects such as user experience, cognitive load, presence, and the impact of simulator sickness on users. Researchers discuss methodologies for designing realistic and effective training scenarios within VR flight simulators. This includes considerations for creating engaging content, mission scenarios, and virtual environments.

Some literature explores the development of collaborative and multiplayer features in VR flight simulators. This allows multiple users to interact within the same virtual airspace, enabling team training scenarios and shared learning experiences.

Evaluation metrics for VR flight simulator projects are a common focus. Researchers propose and analyze metrics for assessing training effectiveness, user performance, and the overall quality of the simulator experience.

VR flight simulators have applications beyond training, such as entertainment, research, and public outreach. The literature discusses these alternative uses and the challenges associated with adapting VR flight simulators for diverse purposes.

Many publications reflect on the challenges faced in developing VR flight simulators and propose future directions for research. Common challenges include hardware limitations, simulator sickness mitigation, and the need for standardization in VR training.

Pre-2010:

Look for early academic papers or industry reports discussing the feasibility of VR technology and initial applications in simulations. Researchers like Ivan Sutherland and Thomas Furness laid the groundwork for VR concepts.

2010-2014:

Investigate the release of Oculus Rift's development kit and related academic publications or industry reviews during this period. Oculus Rift's Kickstarter campaign in 2012 marked a significant turning point.

2015-2017:

Explore academic journals, conference proceedings, and publications discussing the consumer release of major VR headsets, advancements in immersive technologies, and the initial surge of interest in VR flight simulations for gaming and entertainment.

2018-2019:

Investigate research articles and industry reports on the refinement of VR flight simulators, particularly focusing on realism, reducing simulator sickness, and improving the user experience. Hand-tracking controllers and enhanced user interfaces are noteworthy advancements during this period.

2020-2021:

Look for academic papers and industry publications that discuss the continued growth of VR flight simulation, multiplayer options, collaborative training scenarios, and the increasing adoption of VR in educational and professional training contexts.

CHAPTER - 3

3. SOFTWARE REQUIREMENTS SPECIFICATION

In this section we look to describe in detail the requirements we seek from our project to be efficient. This description is divided into 4 parts, Functional Requirement Specification, Performance Requirements, Software Requirements, and Hardware Requirements. In Functional Requirement Specification we identify different modules of our system and express the role performed by each module. In Performance Requirements, we broadly outline the requirements in terms of performance from the system. In Software and Hardware Requirements we label the minimum requirements needed for running the project.

3.1 Functional requirement analysis

Functional requirements explain what has to be done by identifying the necessary task, action or activity that must be accomplished. Functional requirements analysis will be used as the top- level functions for functional analysis.

3.2 User requirements analysis

User Requirements Analysis is the process of determining user expectations for a new or modified product. These features must be quantifiable, relevant and detailed. The main user requirements of our project are as follows:

- Internet Facility/ LAN Connection
- CPU i5+
- RAM 8 or 16 GB
- Memory 128 GB
- Oculus Quest 2

3.3 Input design:

Input design is a part of overall system design. The main objective during the input design is as given below:

- To produce a cost-effective method of input.
- To achieve the highest possible level of accuracy.
- To ensure that the input is acceptable and understood by the user.

3.4 Input types:

It is necessary to determine the various types of inputs. Inputs can be categorized as follows:

- External inputs, which are prime inputs for the system.
- Internal inputs, which are user communications with the system.
- Operational, which are the computer department's communications to the system?
- Interactive, which are inputs entered during a dialogue.

3.5 Output design:

Outputs from computer systems are required primarily to communicate the results of processing to users. They are also used to provide a permanent copy of the results for later consultation. The various types of outputs in general are:

- External Outputs, whose destination is outside the organization.
- Internal Outputs whose destination is within the organization.
- User's main interface with the computer.
- Operational outputs whose use is purely within the computer department.
- Interface outputs, which involve the user in communicating directly.

CHAPTER - 4

4. SYSTEM DESIGN

4.1 Introduction to UML

Software design sits at the technical kernel of the software engineering process and is applied regardless of the development paradigm and area of application. Design is the first step in the development phase for any engineered product or system. The designer's goal is to produce a model or representation of an entity that will later be built. Once system requirements have been specified and analyzed, system design is the first of the three technical activities - design, code and test that is required to build and verify software. The importance can be stated with a single word "Quality". Design is the place where quality is fostered in software development. Design provides us with representations of software that can assess quality.

Design is the only way that we can accurately translate a customer's view into a finished software product or system. Software design serves as a foundation for all the software engineering steps that follow. Without a strong design we risk building an unstable system – one that will be difficult to test, one whose quality cannot be assessed until the last stage. The purpose of the design phase is to plan a solution of the problem specified by the requirement document. This phase is the first step in moving from the problem domain to the solution domain. In other words, starting with what is needed, design takes us toward how to satisfy the needs.

The design of a system is perhaps the most critical factor affecting the quality of the software; it has a major impact on the later phase, particularly testing, maintenance. The output of this phase is the design document. This document is similar to a blueprint for the solution and is used later during implementation, testing and maintenance.

The design activity is often divided into two separate phases: System Design and Detailed Design. System Design also called top-level design aims to identify the modules that should be in the system, the specifications of these modules, and how they interact with each other to produce the desired results.

At the end of the system design all the major data structures, file formats, output formats, and the major modules in the system and their specifications are decided. During Detailed Design, the internal logic of each of the modules specified in system design is decided.

In other words, in system design the attention is on what components are needed, while in detailed design how the components can be implemented in software is the issue. Design is concerned with identifying software components specifying relationships among components. Specifying software structure and providing a blueprint for the document phase. Modularity is one of the desirable properties of large systems.

It implies that the system is divided into several parts. In such a manner, the interaction between parts is minimal and clearly specified. During the system design activities, Developers bridge the gap between the requirements specification, produced during requirements elicitation and analysis, and the system that is delivered to the user. Design is the place where the quality is fostered in development. Software design is a process through which requirements are translated into a representation of software.

4.2 UML Design:

UML combines best techniques from data modeling (entity relationship diagrams), business modeling (workflows), object modeling, and component modeling. It can be used with all processes, throughout the software development life cycle, and across different implementation technologies. UML has synthesized the notations of the Bloch method, the Object-modeling technique (OMT) and Object-oriented software engineering (OOSE) by fusing them into a single, common and widely usable modeling language. UML aims to be a standard modeling language which can model concurrent and distributed systems.

4.2.1 Use case Diagram:

Use case diagrams model the functionality of a system using actors and use cases. Use cases are services or functions provided by the system to its users.

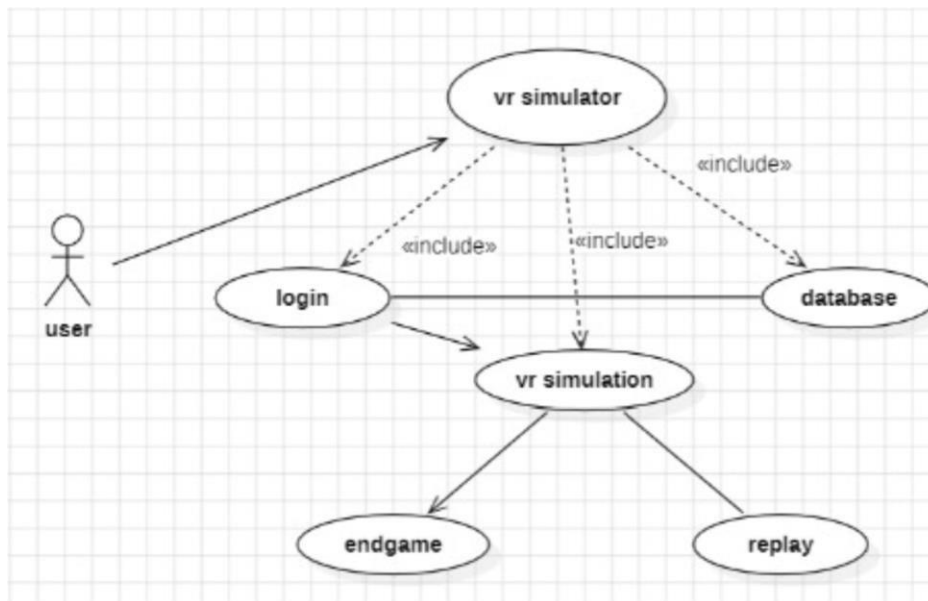


Figure.1: Use Case Diagram

4.2.2 Class diagram:

Class diagram is a static diagram. It represents the static view of an application. Class diagram is not only used for visualizing, describing, and documenting different aspects of a system but also for constructing executable code of the software application.

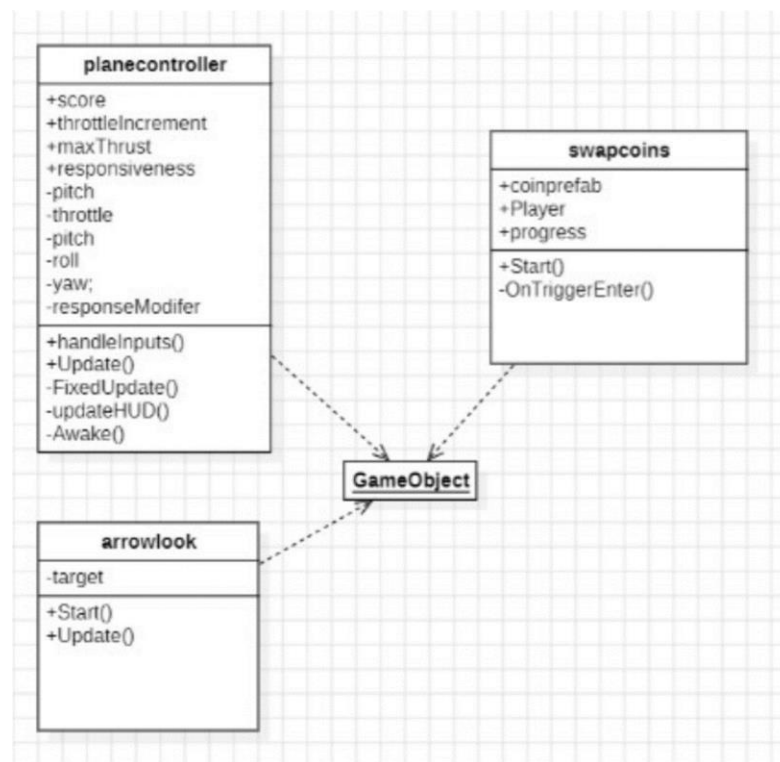


Figure.2: Class Diagram

CHAPTER - 5

5. METHODOLOGIES

5.1 Problems with Existing System:

There are several common problems associated with flight simulation systems. Achieving a high level of realism in flight simulation can be challenging. Simulators may struggle to replicate the intricate details of real-world flight dynamics, weather conditions, and aircraft behavior.

High-fidelity flight simulation often requires powerful hardware, including advanced graphics cards, processors, and memory. Some users may find it challenging to afford or set up the necessary equipment for an optimal experience.

Ensuring that flight simulators can run effectively across a range of hardware configurations is a common challenge. Developers aim to strike a balance between realistic graphics and performance on a variety of systems.

Realistic weather simulation is crucial for an immersive flight experience. However, accurately simulating complex weather patterns, including turbulence and wind shear, can be computationally demanding.

Keeping the simulator's navigation databases and scenery up to date with real-world changes is essential for accurate training and simulation. Frequent updates are required to reflect changes in airports, airspaces, and navigation procedures.

Building and maintaining high-fidelity flight simulators can be expensive, limiting accessibility for individual users or small organizations. The cost of licenses for realistic aircraft and other add-ons can also be a barrier.

While virtual reality (VR) has made significant strides in recent years, some flight simulators may not fully leverage VR technology, limiting the immersive experience for users who prefer this mode of interaction. Flight simulators, especially those aimed at professional training, may have a steep learning curve.

The complexity of operating the simulator and interpreting various instruments can be a challenge for new users. Ensuring a seamless and realistic multiplayer experience, including realistic air traffic control and communication between users, can be challenging.

Network latency and synchronization issues can impact the overall experience. Dependency on third-party developers for aircraft, scenery, and other add-ons can lead to inconsistencies in quality and support. A strong and active community is essential for the ongoing development and improvement of the simulator.

5.2 Proposed System:

The VR Flight Simulator project will have the following features:

- **Realistic aircraft:** A single flight 3D object with collider components to enable collision detection with other objects in the scene.
- **Terrain:** The simulator offers an environment for the user to fly in, such as a city and mountains.
- **Realistic Controls:** C# is used to control the movement and physics of the aircraft, allowing for realistic flight simulation.
- **Integration:** The VR Flight Simulator project will have a client-server architecture.

Creating terrain in Unity is a straightforward process that involves using the Terrain object in the Unity editor. The Terrain object allows you to create custom terrain by adjusting parameters such as the height, texture, and details of the terrain. For the VR Flight Simulator project, the terrain will be created using a combination of the built-in Unity terrain tools and custom assets. Different textures and materials will be used to create the appearance of terrain.

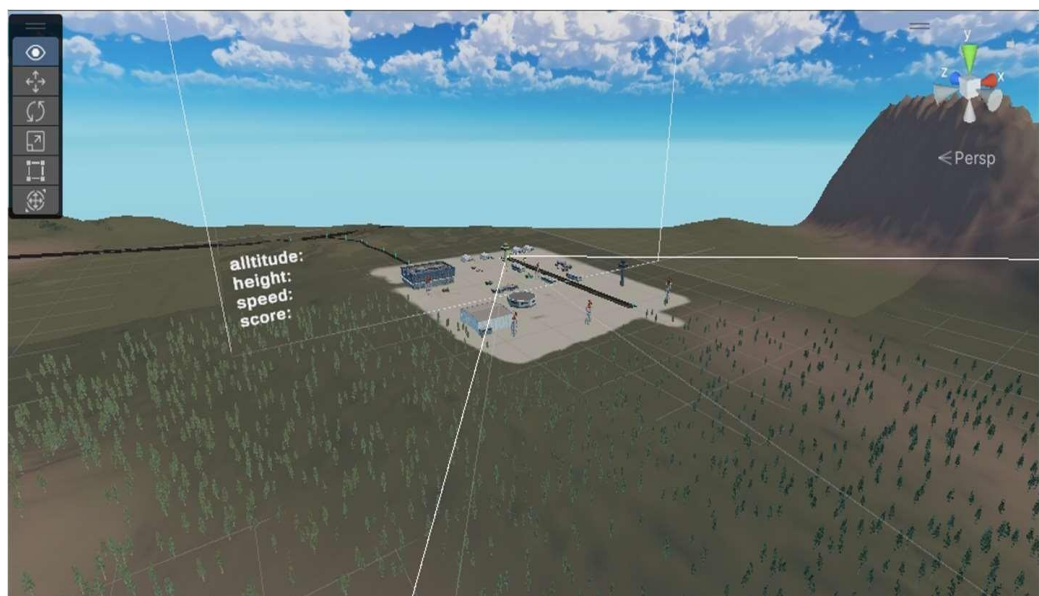


Figure.3: Terrain

To add a flight 3D object in Unity, you can import a 3D model. Once you have your 3D model, you can add it to your project by dragging and dropping it into the Unity editor. From there, you can adjust its position, rotation, and scale to fit your project's needs.

To make the flight 3D object interact with other objects in the scene, you will need to add collider components to it. Collider components define the shape and size of an object's collision area and allow it to interact with other objects in the scene. Unity provides several types of collider components, such as Box Collider, Sphere Collider, and Mesh Collider, which can be added to 3D objects depending on their shape and size. In the VR Flight Simulator project, C# is used to control the movement of the aircraft, which includes simulating realistic physics and handling user input. The movement control code uses various C# classes and methods provided by Unity, such as Rigid body, Transform, and Input, to manipulate the aircraft's position, rotation, and velocity.

The C# code also handles user input, such as keyboard controls, to control the aircraft's movement. This includes detecting input events and translating them into movements of the aircraft, such as changing its speed, direction, or altitude.

Additionally, the movement control code includes error handling and safety checks to ensure that the aircraft's movement remains within realistic and safe bounds during flight.

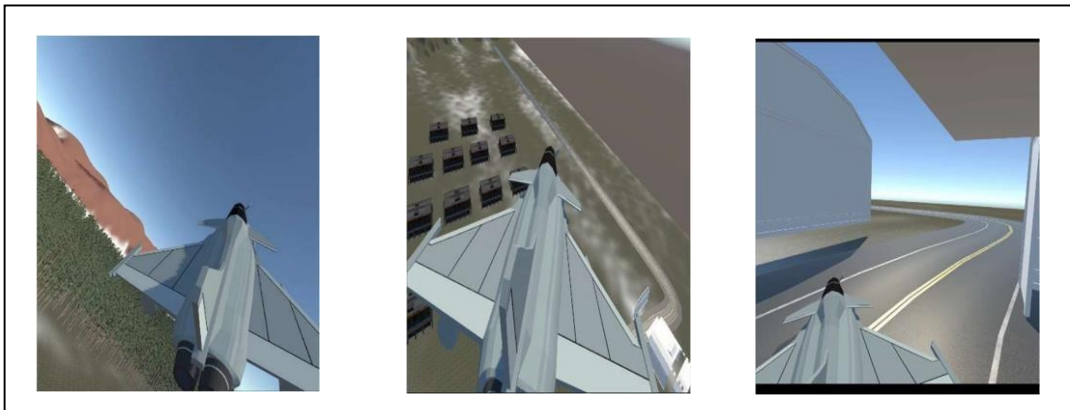


Figure 4. Simulation Clips

CHAPTER - 6

6. IMPLEMENTATION

CODE SNIPPETS:

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public class PlaneController : MonoBehaviour
{
    public float throttleIncrement = 0.1f;
    public float maxThrottle = 200f;
    public float responsiveness = 10f;
    private float throttle;
    private float roll;
    private float pitch;
    private float yaw;
    private float responseModifier
    {
        get
        {
            return (rb.mass / 10f) * responsiveness;
        }
    }
    Rigidbody rb;
```

```
private void Awake()
{
    rb = GetComponent<Rigidbody>();
}

private void HandleInputs()
{
    /*roll = Input.GetAxis("Roll");
    pitch = Input.GetAxis("Pitch");
    yaw = Input.GetAxis("Yaw");*/

    roll =
OVRInput.Get(OVRInput.Axis2D.PrimaryThumbstick).x;
    pitch =
OVRInput.Get(OVRInput.Axis2D.PrimaryThumbstick).y;
    yaw =
OVRInput.Get(OVRInput.Axis2D.SecondaryThumbstick).x;

    throttle = Mathf.Clamp(throttle +
OVRInput.Get(OVRInput.Axis2D.SecondaryThumbstick).y *
throttleIncrement, 0f, 100f);

    //Handle throttle value being sure to clamp it
    between 0 and 100.
```

```
    if (Input.GetKey(KeyCode.Space) ||
        OVRInput.Get(OVRInput.Button.One)) throttle +=
        throttleIncrement;

        else if (Input.GetKey(KeyCode.LeftControl) ||
            OVRInput.Get(OVRInput.Button.Two)) throttle -=
            throttleIncrement;

        throttle = Mathf.Clamp(throttle, 0f, 100f);
    }

    // Start is called before the first frame update
    void Start()
    {

    }

    // Update is called once per frame
    private void Update()
    {
        HandleInputs();
    }

    private void FixedUpdate()
    {
        rb.AddForce(transform.forward * maxThrottle *
            throttle);

        rb.AddTorque(transform.up * yaw *
            responseModifier);
    }
}
```

```
        rb.AddTorque(transform.right * pitch * responseModifier);  
        rb.AddTorque(transform.forward * roll *  
responseModifier);  
    }  
}
```


CHAPTER -7

7. SCREENSHOTS

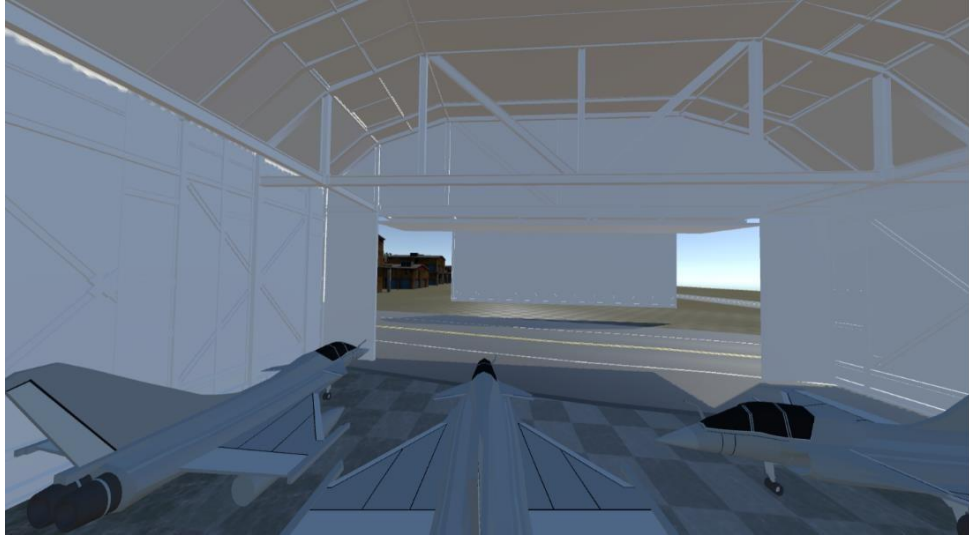


FIGURE 5: HANGAR

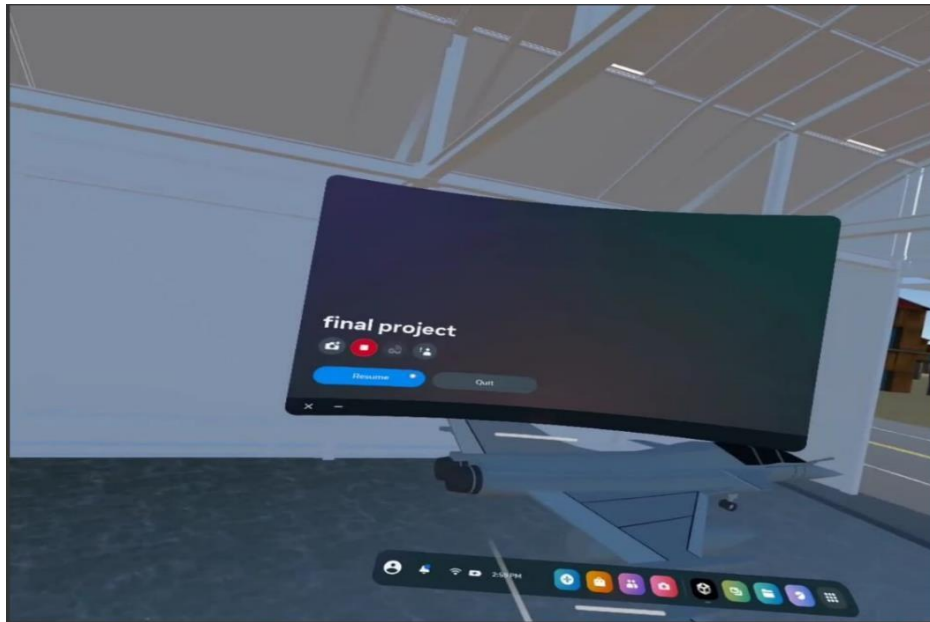


FIGURE 6: OCCULUS UI



FIGURE 7: FLIGHT DYNAMICS



FIGURE 8: FLIGHT DYNAMICS

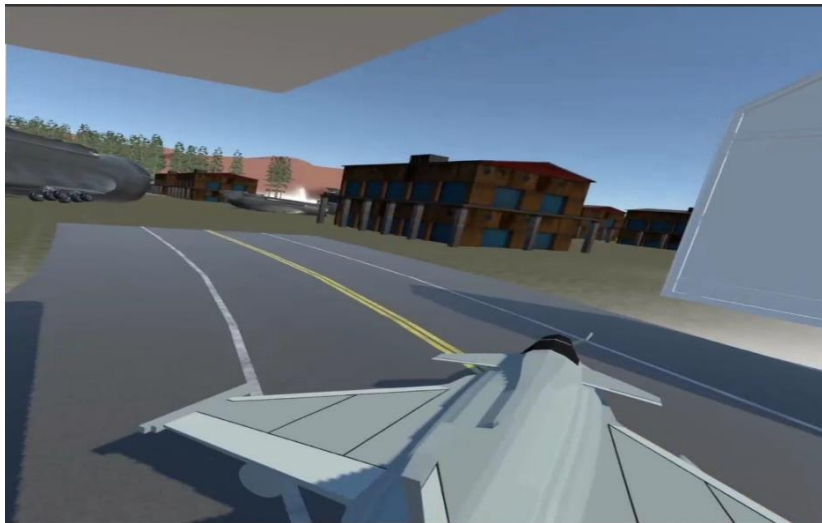


FIGURE 9: FLIGHT RUNWAY

CHAPTER - 8

8.**TESTING****8.1 Introduction to Testing**

Testing is the process of evaluating a system or its component(s) with the intent to find whether it satisfies the specified requirements or not. Testing is executing a system in order to identify any gaps, errors, or missing requirements in contrary to the actual requirements.

According to ANSI/IEEE 1059 standard, Testing can be defined as - A process of analyzing a software item to detect the differences between existing and required conditions (that is defects/errors/bugs) and to evaluate the features of the software item. It depends on the process and the associated stakeholders of the project. In most cases, the following professionals are involved in testing a system within their respective capacities:

- Software Tester
- Software Developer
- Project Lead/Manager
- End User

Levels of testing include different methodologies that can be used while conducting software testing. The main levels of software testing are:

- Functional Testing
- Non-functional Testing

Functional Testing:

This is a type of black box testing that is based on the specifications of the software that is to be tested. The application is tested by providing input and then the results are examined that need to conform to the functionality it was intended for. Functional testing of a software is conducted on a complete, integrated system to evaluate the system's compliance with its specified requirements.

8.2 Software Testing Life Cycle:

The process of testing a software in a well-planned and systematic way is known as software testing lifecycle (STLC). Different organizations have different phases in STLC however generic Software Test Life Cycle (STLC) for waterfall development model consists of the following phases:

1. Requirements Analysis
2. Test Planning
3. Test Analysis
4. Test Design

Requirements Analysis:

In this phase testers analyze the customer requirements and work with developers during the design phase to see which requirements are testable and how they are going to test those requirements. It is very important to start testing activities from the requirements phase itself because the cost of fixing defect is very less if it is found in requirements phase rather than in future phases.

Test Planning:

In this phase all the planning about testing is done like what needs to be tested, how the testing will be done, test strategy to be followed, what will be the test environment, what test methodologies will be followed, hardware and software availability, resources, risks etc. A high-level test plan document is created which includes all the planning inputs mentioned above and circulated to the stakeholders.

Test Analysis:

After test planning phase is over test analysis phase starts, in this phase we need to dig deeper into project and figure out what testing needs to be carried out in each SDLC phase. If automation needs to be done for software product, how much time will it take to automate, and which features need to be automated. Nonfunctional testing areas are also analyzed and defined in this phase.

Test Design:

In this phase various black-box and white-box test design techniques are used to design the test cases for testing, testers start writing test cases by following those design techniques, if automation testing needs to be done then automation scripts also need to be written in this phase.

8.3 White Box Testing:

We'll be using white box testing for the project in hand since we're using python programming language. White Box Testing is software testing technique in which internal structure, design and coding of software are tested to verify flow of input-output and to improve design, usability, and security. In white box testing, code is visible to testers, so it is also called Clear box testing, Open box testing, Transparent box testing, Code-based testing and Glass box testing.

What do you verify in White Box Testing?

White box testing involves the testing of the software code for the following:

- Internal security holes
- Broken or poorly structured paths in the coding processes.
- The flow of specific inputs through the code.

Expected output

The functionality of conditional loops

Testing of each statement, object, and function on an individual basis.

How do you perform White Box Testing?

To give you a simplified explanation of white box testing, we have divided it into two basic steps. This is what testers do when testing an application using the white box testing technique:

Step 1: Understand the source code the first thing a tester will often do is learn and understand the source code of the application. Since white box testing involves the testing of the inner workings of an application, the tester must be very

knowledgeable in the programming languages used in the applications they are testing. Step 2: Create test cases and execute the second basic step to white box testing involves testing the application's source code for proper flow and structure. One way is by writing more code to test the application's source code.

White Box Testing Techniques:

A major White box testing technique is Code Coverage analysis. Code Coverage analysis eliminates gaps in a Test Case suite. It identifies areas of a program that are not exercised by a set of test cases. Once gaps are identified, you create test cases to verify untested parts of the code, thereby increasing the quality of the software product. There are automated tools available to perform Code coverage analysis. Below are a few coverage analysis techniques

30 Statement Coverage- This technique requires every possible statement in the code to be tested at least once during the testing process of software engineering.

Types of White Box Testing:

White box testing encompasses several testing types used to evaluate the usability of an application, block of code or specific software package. They are listed below:

Unit Testing: It is often the first type of testing done on an application. Unit testing is performed on each unit or block of code as it is developed. Unit Testing is essentially done by the programmer. As a software developer, you develop a few lines of code, a single function or an object and test it to make sure it works before continuing. Unit Testing helps identify a majority of bugs, early in the software development lifecycle. Bugs identified in this stage are cheaper and easy to fix.

Testing for Memory Leaks: Memory leaks are leading causes of slower running applications. A QA specialist who is experienced at detecting memory leaks is essential in cases where you have a slow running software application. Apart from above, a few testing types are part of both black box and white box testing.

White Box Penetration Testing: In this testing, the tester/developer has full information of the application's source code, detailed network information, IP addresses involved and all server information the application runs on. The aim is to attack the code from several angles to expose security threats.

Advantages of White Box Testing:

- Code optimization by finding hidden errors.
- White box test cases can be easily automated.
- Testing is more thorough as all code paths are usually covered.
- Testing can start early in SDLC even if GUI is not available.

Disadvantages of white box:

- White box testing can be quite complex and expensive.
- Developers who usually execute white box test cases detest it. The white box testing by developers is not detailed and can lead to production errors.
- White box testing requires professional resources, with a detailed understanding of programming and implementation.

8.4 What is Black Box Testing?

Black box testing is defined as a testing technique in which functionality of the Application under Test (AUT) is tested without looking at the internal code structure, implementation details and knowledge of internal paths of the software. This type of testing is based entirely on software requirements and specifications. In Black Box Testing we just focus on inputs and output of the software system without bothering about internal knowledge of the software program.

How to do Black Box Testing:

Here are the generic steps followed to carry out any type of Black Box Testing.

- Initially, the requirements and specifications of the system are examined.
- Tester determines expected outputs for all those inputs.
- Software tester constructs test cases with the selected inputs. The test cases are executed.
- The software tester compares the actual outputs with the expected outputs.

Types of Black Box Testing:

There are many types of Black Box Testing but the following are the prominent ones:

Functional testing:

This black box testing type is related to the functional requirements of a system; it is done by software testers.

Non-functional testing:

This type of black box testing is not related to testing of specific functionality, but non-functional requirements such as performance, scalability, and usability.

Regression testing:

Regression Testing is done after code fixes, upgrades or any other system maintenance to check the new code has not affected the existing code.

Tools used for Black Box Testing:

Tools used for Black box testing largely depend on the type of black box testing you are doing.

- For Functional/Regression Tests you can use -QTP, Selenium
- For Non-Functional Tests, you can use – LoadRunner, J-meter.

Black Box Testing Techniques:

Following are the prominent Test Strategy amongst the many used in Black box Testing:

- Equivalence Class Testing: It is used to minimize the number of possible test cases to an optimum level while maintaining reasonable test coverage.
- Boundary Value Testing: Boundary value testing is focused on the values at boundaries.

CHAPTER - 9

9. FUTURE ENHANCEMENTS

Advanced Aircraft Models: Integrate more realistic aircraft models with detailed cockpits and exterior designs. You could include a variety of aircraft types such as commercial airliners, military jets, helicopters, and vintage planes. Each aircraft should have unique handling characteristics and cockpit layouts, providing users with a diverse range of flying experiences.

Dynamic Weather System: Implement a dynamic weather system that simulates various weather conditions such as rain, snow, fog, wind, and turbulence. This would add an extra layer of realism to the simulation and challenge players to adapt their flying skills accordingly. Incorporating real-world weather data APIs could enable the simulator to fetch current weather conditions from around the globe.

Interactive Air Traffic Control (ATC): Create an interactive ATC system that provides realistic communication between pilots and air traffic controllers. Users should be able to request takeoff and landing clearances, receive flight instructions, and navigate through controlled airspace.

Implementing voice recognition technology could allow players to communicate with virtual ATC using voice commands, enhancing the immersion of the experience.

Expanded Scenery and Terrain: Expand the virtual world with more detailed scenery and terrain features. Incorporate landmarks, cities, mountains, rivers, and other geographic elements to create a visually stunning and immersive environment. Utilize procedural generation techniques to generate vast expanses of terrain realistically, ensuring that the virtual world feels expansive and diverse.

Multiplayer Support: Introduce multiplayer functionality that allows users to fly together in the same virtual airspace. This could include features such as cooperative missions, air races, formation flying, and air traffic control roles. Implement networking capabilities to synchronize the positions and actions of multiple players, enabling seamless multiplayer experiences.

Flight Training and Tutorials: Develop comprehensive flight training modules and tutorials to help users learn the fundamentals of aviation and master various flying techniques. Incorporate interactive lessons, instructional videos, and guided missions that cover topics such as aircraft controls, navigation, flight planning, and emergency procedures. Provide feedback and performance metrics to track users' progress and proficiency.

Customization and Modding Support: Allow users to customize their experience by modifying aircraft, scenery, weather conditions, and other aspects of the simulation. Provide tools and documentation for modders to create and share their content, fostering a vibrant community of user-generated addons and enhancements. Implement support for downloadable content (DLC) and third-party plugins to extend the simulator's capabilities further.

Virtual Reality Improvements: Continuously optimize and refine the VR experience to improve performance, comfort, and immersion. Experiment with advanced VR rendering techniques, such as foveated rendering, dynamic resolution scaling, and VR-specific shaders, to enhance visual quality and maintain consistent frame rates. Integrate support for the latest VR hardware devices and controllers to ensure compatibility with a wide range of VR setups.

By incorporating these future enhancements into your VR flight simulator project, you can create a more realistic, immersive, and engaging experience for users, offering them a compelling simulation of flight in virtual reality.

CHAPTER – 10

10. CONCLUSION

In conclusion, the development of a VR flight simulator using Unity represents a significant achievement in the realm of virtual reality gaming and simulation. Throughout the course of this project, we have embarked on a journey to create an immersive and realistic experience that allows users to take to the skies and explore the wonders of flight from the comfort of their own home.

From the initial stages of conceptualization to the final implementation, this project has been guided by a vision of delivering a captivating and authentic flight simulation experience. By leveraging the power of Unity's robust game development tools and the immersive capabilities of virtual reality technology, we have been able to create a compelling environment that transports users into the cockpit of a variety of aircraft, allowing them to experience the thrill of flying firsthand. One of the key highlights of this project is the integration of realistic aircraft models with detailed cockpits and exterior designs. By meticulously recreating the look and feel of various aircraft types, we have been able to provide users with a diverse range of flying experiences, from piloting commercial airliners to maneuvering agile military jets. Furthermore, the implementation of dynamic weather systems and interactive air traffic control adds an extra layer of realism to the simulation, challenging players to adapt their flying skills to different environmental conditions and navigate through controlled airspace with precision and accuracy. The expansion of the virtual world with detailed scenery and terrain features enhances the sense of immersion and exploration, allowing users to fly over iconic landmarks, bustling cities, majestic mountains, and scenic landscapes. Moreover, the addition of multiplayer support enables users to fly together in the same virtual airspace, fostering collaboration, competition, and social interaction among players.

Whether it's cooperating on missions, participating in air races, or simply enjoying the freedom of flight together, multiplayer functionality adds a new dimension to the VR flight simulator experience.

In addition to providing entertainment and enjoyment, this project also serves as a valuable educational tool, offering users the opportunity to learn about the principles of aviation, flight mechanics, navigation, and more.

The inclusion of comprehensive flight training modules and tutorials ensures that users can develop their flying skills and knowledge at their own pace, making the simulator accessible to enthusiasts of all levels of experience.

Looking ahead, there are endless possibilities for future enhancements and refinements to further elevate the VR flight simulator experience. Whether it's incorporating advanced rendering techniques, expanding the range of aircraft and scenery, or integrating emerging technologies such as augmented reality and artificial intelligence, the potential for innovation and creativity is limitless.

In conclusion, the development of a VR flight simulator using Unity represents a testament to the power of technology to inspire, educate, and entertain. By combining cutting-edge software development with the immersive capabilities of virtual reality, we have created an experience that truly takes flight simulation to new heights.

CHAPTER – 11

11. REFFERENCES

1. Unity Technologies. (n.d.). Unity - Game Engine. [Website]. <https://unity.com/>
- Unity 3D is the game engine used for the flight simulator development.
2. Oculus Developer Documentation. (n.d.). Oculus. [Website].
<https://developer.oculus.com/>
3. Google VR Documentation. (n.d.). Google VR. [Online]. Available:
<https://developers.google.com/vr>
4. Unity Asset Store. (n.d.). Unity Asset Store. [Online]. Available:
<https://assetstore.unity.com/>
5. Brinkmann, A., Schluse, M., & Dittmar, A. (2019). Design and Development of a Virtual Reality Flight Simulator for Educational Purposes. In International Conference on Interactive Collaborative Learning (pp. 381-391). Springer, Cham.
6. Microsoft Docs. (n.d.). Microsoft Flight Simulator. [Website].
<https://docs.microsoft.com/en-us/windows/mixed-reality/flightsimulator>
Microsoft Flight Simulator serves as a reference for the development of realistic flight simulation experiences.
7. X-Plane. (n.d.). X-Plane 11. [Website]. <https://www.x-plane.com/>
X-Plane 11 is a well-known flight simulator used in aviation training and research.
8. FlightGear Flight simulator. [Website]. <https://www.flightgear.org/>
FlightGear is an open-source flight simulator known for community-driven development and a wide range of aircraft and scenery.
9. Lockheed Martin. (n.d.). Prepar3D. [Website]. <https://www.prepar3d.com/>

Lockheed Martin's Prepar3D is used in professional aviation training and is relevant for advanced flight simulation

10. Looking Glass Studios. (n.d.). Flight Unlimited III.

11. Bowman, D. A., McMahan, R. P., & Ragan, E. D. (2017